AI Coding Quick Reference — Rux UI / TripBoard

> Classify first. Configure second. Escalate only for a reason.

> Use the least expensive configuration that reliably produces the correct result.

Use this card for day-to-day decisions. See the full operating guide for rationale, detailed guardrails, and reusable prompts.

1. Classify the Task

Type	Signal	Working posture
1 Fast	Tiny, obvious, localized edit	Low reasoning; focused check
2 Routine	Normal feature following an existing pattern	Medium reasoning; implement and test
3 Hard	Root cause unclear, cross-file state, difficult review	High reasoning; reproduce/trace first
4 Plan	Decide architecture, schema, migration, or boundaries	High reasoning; read-only Plan mode
5 Large	Broad migration or independent workstreams	High–max; staged checkpoints
+V Visual	Appearance or interaction determines success	Add rendered visual verification
+R Risk	Production data, auth, secrets, permissions, destructive/breaking work	Reduce autonomy; require proposal and approval

2. Pick the Configuration

Type	Codex	Claude Code	Cline	Mode
1	Luna · low	Sonnet · low	ClinePass Flash · low	Act/edit
2	Terra · medium	Sonnet · medium	ClinePass Pro · medium	Act/edit
3	Sol · high	Fable · high	ClinePass Pro · high	Manual/checkpoints
4	Sol · high	Fable · high	ClinePass Pro · high	Plan/read-only
5	Sol · max	Fable · high–max	ClinePass Pro · max	Staged autonomy

Everyday default: **Type 2**. Move down for trivial work. Move up only when the task or evidence justifies it.

Multi-agent/Ultra is for Type 5 work that genuinely divides into independent workstreams. It is not a substitute for a clear scope.

Model names and availability change. Use the current equivalent tier offered by each platform.

3. Apply Modifiers

Modifier	Add to the normal posture
V Visual	Read `SKILL.md`; inspect `css/tokens.css` when relevant; reuse Rux primitives; render narrow/wide and light/dark states
R Risk	Inspect first; use Manual/Guarded mode; state blast radius and rollback; require approval before destructive/production execution

Hard task does not mean high autonomy. Often the right configuration is **strong model + high reasoning + low autonomy**.

4. Load Only the Needed Context

Task	Read in this order
UI/CSS	`SKILL.md` → `css/tokens.css` when relevant → target markup/style
Application behavior	Target `js/` module or `index.html` section → direct callers/dependencies
Data/Supabase	Relevant `js/data/` module → related `supabase/` SQL → affected consumers
Hard debugging	Failure evidence → failing module → caller/callee chain as needed

Do not read `SKILL.md` for unrelated work. Do not scan the entire repository for a narrow task.

5. Escalate Deliberately

```
Fix context/permissions/criteria
→ add failure evidence
→ increase reasoning
→ stronger model
→ Plan mode for unresolved architecture
→ multi-agent/Ultra only for independent workstreams
```

Do not retry the same failed approach repeatedly.

6. Repository Guardrails

- Make the smallest coherent change; preserve public contracts and avoid unrelated refactors.
- Planning is read-only; Supabase is live; production/destructive execution needs explicit authorization.
- For bugs, reproduce or trace first and verify the original failure path afterward.
- Run `node --test tests/<file>.test.mjs` for focused coverage or `npm test` for the full suite.
- Review the final diff, run `git diff --check`, and report anything unverified.

7. Prompt Formula

```
TYPE: [1–5][V][R]
GOAL: What must be true when complete?
CONTEXT: Which subsystem/files matter?
CONSTRAINTS: What must not change?
SCOPE: What may the agent modify?
VERIFICATION: What evidence proves correctness?
```

Example:

```
TYPE: 2V
GOAL: Add the TripBoard location row to assignment cards.
CONTEXT: Read SKILL.md, the assignment-card implementation, and
css/tokens.css.
CONSTRAINTS: Reuse Rux primitives; preserve cards without a location.
SCOPE: Modify only required assignment-card files.
VERIFICATION: Run focused tests and inspect narrow/wide layouts in both
themes.
```

Personal reference last reviewed: 2026-08-11. Keep platform defaults here synchronized with the full operating guide; repository behavior remains in `AGENTS.md`, `CLAUDE.md`, and `.cline/rules/project.md`.